ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

Departamento de Ingeniería

Trabajo Practico 01

Ejercicio 2: Coloracion de Grafos con k Colores (Backtracking)

Asignatura: Analisis y Diseño de Algoritmos

Profesor: Carlos Alberto Álvarez Henao

Integrantes: Andrés Felipe Vélez Álvarez
Sebastian Salazar Henao
Nathalia Valentina Cardoza Azuaje

Fecha: 5 de abril de 2026

Universidad EAFIT

Índice

1. Descripcion Teorica del Problema **2**
1.1. Planteamiento . . . 2
1.2. El Numero Cromatico . . . 2
1.3. Representacion del Grafo: Matriz de Adyacencia . . . 3

2. Enfoques de Solucion **3**
2.1. Backtracking . . . 3
2.2. Fuerza Bruta . . . 4
2.3. Comparacion de Enfoques . . . 4

3. Pseudocodigo **4**
3.1. Backtracking . . . 4
3.2. Fuerza Bruta . . . 5

4. Implementacion en C++ **5**

5. Ejemplos de Entrada y Salida **8**
5.1. Ejemplo 1 — Ciclo C4 (n=4, k=3) . . . 8
5.2. Ejemplo 2 — Triangulo K3 (n=3, k=2) — Sin solucion . . . 9
5.3. Ejemplo 3 — Grafo completo K7 (n=7, k=4) — Diferencia dramatica . . . 10

6. Comparacion de Tiempos de Ejecucion **10**

7. Analisis de Complejidad **11**
7.1. Complejidad Temporal . . . 11
7.1.1. Fuerza Bruta . . . 11
7.1.2. Backtracking . . . 11
7.2. Complejidad Espacial . . . 12

8. Respuestas a las Preguntas Guia **12**
8.1. ¿Cuantos nodos explora el Backtracking vs la Fuerza Bruta? . . . 12
8.2. ¿Que tipo de grafo maximiza o minimiza la poda? . . . 12
8.3. ¿Que cambio hacer para saber si existe al menos una k-coloracion? . . . 13
8.4. Relacion entre la k-coloracion y el Numero Cromatico . . . 13

9. Conclusiones **13**

1. Descripcion Teorica del Problema

1.1. Planteamiento

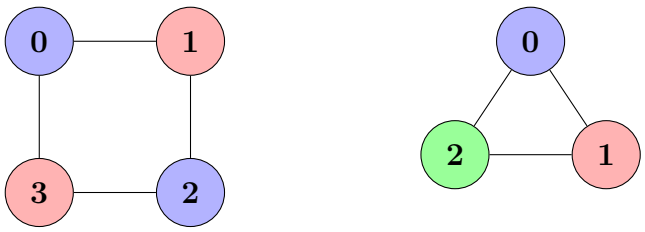
Imagina un mapa donde hay varios paises vecinos y se quiere pintarlos de tal forma que dos paises que compartan frontera nunca tengan el mismo color. Ese es exactamente el concepto detras de la **coloracion de grafos**.

En terminos formales, un **grafo** es un conjunto de puntos llamados **vertices** y lineas que los conectan llamadas **aristas**. La coloracion de grafos consiste en asignar un color a cada vertice de manera que ningun par de vertices conectados por una arista tenga el mismo color. Cuando se tienen exactamente k colores disponibles, se habla de una **k-coloracion valida**, y la condicion que debe cumplirse es:

$$\forall\,(u,v) \in E\colon \mathrm{color}[u] \neq \mathrm{color}[v] \tag{1}$$

Este problema tiene aplicaciones practicas muy concretas:

- **Horarios de examenes:** cada materia es un vertice; si dos materias tienen estudiantes en comun se conectan con una arista. Colorear el grafo equivale a asignar franjas horarias de modo que ningun estudiante tenga dos examenes al mismo tiempo.
- **Frecuencias de radio:** dos antenas cercanas no pueden usar la misma frecuencia. Cada antena es un vertice y la frecuencia asignada es el color.
- **Compiladores:** las variables que se usan al mismo tiempo no pueden compartir el mismo registro del procesador. Cada variable es un vertice; si dos se usan simultaneamente se conectan, y el color representa el registro asignado.



C_4 con 2 colores K_3 con 3 colores

Figura 1: Ejemplos de coloraciones validas: ningun par de vertices vecinos comparte color.

1.2. El Numero Cromatico

El **numero cromatico** $\chi(G)$ responde a la pregunta: *¿con cuantos colores como minimo se puede colorear correctamente este grafo?*

Algunos ejemplos intuitivos:

- Un grafo sin aristas necesita solo 1 color: $\chi = 1$.
- Cualquier ciclo de longitud par (cuadrado, hexagono...) se puede colorear alternando 2 colores: $\chi = 2$.

- Un triangulo, donde los tres vertices estan todos conectados entre si, obliga a usar 3 colores distintos: $\chi = 3$.
- Un grafo donde todos estan conectados con todos (K_n) necesita exactamente n colores: $\chi = n$.

La relacion con el problema de este ejercicio es directa: el grafo admite una k-coloracion valida **si y solo si** $\chi(G) \leq k$. El algoritmo desarrollado puede calcular $\chi(G)$ probando con $k = 1, 2, 3, \ldots$ hasta encontrar la primera k para la que existe solucion.

1.3. Representacion del Grafo: Matriz de Adyacencia

Para trabajar con el grafo en el programa se usa una **matriz de adyacencia**: una tabla cuadrada de $n \times n$ donde la posicion (i, j) vale 1 si hay una arista entre el vertice i y el vertice j, y 0 si no la hay. Como el grafo es no dirigido, la matriz siempre es simetrica ($A[i][j] = A[j][i]$).

Por ejemplo, el ciclo C_4 se representa asi:

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

La fila i indica con cuales vertices esta conectado el vertice i.

2. Enfoques de Solucion

2.1. Backtracking

El **backtracking** (vuelta atras) construye la solucion asignando un color a cada vertice uno por uno. La clave del metodo esta en que, antes de continuar hacia el siguiente vertice, verifica si la asignacion actual genera algun conflicto con los vecinos ya coloreados. Si lo hay, *retrocede* (backtrack) y prueba el siguiente color disponible, sin explorar ese camino que ya se sabe invalido. A esta eliminacion anticipada se le llama **poda**.

Proceso paso a paso:

1. Se intenta asignar un color $c \in \{1, \ldots, k\}$ al vertice v.
2. Se verifica que ningun vecino ya coloreado tenga el color c (`es_seguro`).
3. Si hay conflicto: se descarta ese color y se prueba el siguiente — poda inmediata.
4. Si no hay conflicto: se asigna y se avanza al vertice $v + 1$.
5. Cuando $v = n$: todos los vertices estan coloreados; se cuenta como solucion valida.
6. Al regresar de la recursion se deshace la asignacion (backtrack) para explorar otras combinaciones.

2.2. Fuerza Bruta

La **fuerza bruta** genera directamente las k^n combinaciones posibles de colores para los n vertices y verifica cada una de ellas. No realiza ninguna poda: aunque los primeros vertices ya presenten un conflicto, el algoritmo sigue enumerando hasta el final antes de descartar esa combinacion.

2.3. Comparacion de Enfoques

Cuadro 1: Diferencias entre Backtracking y Fuerza Bruta

Aspecto	**Backtracking**	**Fuerza Bruta**
Espacio de busqueda	Arbol podado	Todas las k^n combinaciones
Poda	Si (conflicto $\Rightarrow$ descarta)	No
Completitud	Si	Si
Correccion	Si	Si
Complejidad peor caso	$O(k^n)$	$O(k^n \cdot n^2)$
Eficiencia practica	Mucho mejor en grafos densos	Inferior

3. Pseudocodigo

3.1. Backtracking

```
leer n, k
leer adj[0..n-1][0..n-1]   // 1 si hay arista, 0 si no
color[0..n-1] = 0           // 0 = sin color asignado
total_soluciones = 0
primera_guardada = falso

// Verifica si el color c es seguro para el vertice v
es_seguro(v, c):
  para cada u en [0..n-1]:
    si adj[v][u] == 1 y color[u] == c:
      retornar falso
  retornar verdadero

// Funcion recursiva principal
bt(v):
  nodos_bt += 1
  si v == n:                        // todos coloreados: solucion
     valida
    total_soluciones += 1
    si !primera_guardada:
      primera_sol = copiar(color)
      primera_guardada = verdadero
    retornar
  para c en [1..k]:
    si es_seguro(v, c):
      color[v] = c
```

```
      bt(v + 1)
      color[v] = 0                    // backtrack: deshacer
         asignacion
    // si NO es seguro: poda, se omite esta rama

bt(0)
si total_soluciones == 0:
  imprimir "No existe una k-coloracion valida"
si no:
  imprimir "Total:", total_soluciones
  imprimir "Primera solucion:", primera_sol
```

Listing 1: Pseudocodigo: Coloracion por Backtracking

3.2. Fuerza Bruta

```
fuerza_bruta():
  total_comb = k^n
  total_fb = 0
  para mask en [0..total_comb-1]:
    nodos_fb += 1
    tmp = mask
    para v en [0..n-1]:
      col[v] = (tmp mod k) + 1        // colores de 1 a k
      tmp = tmp / k                   // division entera
    si verificar_coloracion(col):
      total_fb += 1

// Verifica que ninguna arista tenga ambos extremos del mismo color
verificar_coloracion(col):
  para cada arista (u,v) en E:
    si col[u] == col[v]:
      retornar falso
  retornar verdadero
```

Listing 2: Pseudocodigo: Coloracion por Fuerza Bruta

4. Implementacion en C++

El codigo se organiza en cinco bloques: lectura del grafo, la funcion auxiliar `es_seguro`, el backtracking recursivo `bt`, la fuerza bruta `fuerza_bruta`, y el programa principal con medicion de tiempos y verificacion cruzada.

Un aspecto importante de la implementacion es la **medicion de tiempos**: se usa `duration_cast<nanos` en lugar de milisegundos, y una funcion auxiliar `formato_tiempo` que escala automaticamente la unidad (ns / us / ms / s) segun la magnitud real del tiempo. Esto evita que grafos pequenos siempre muestren `0.0000 ms`.

```
#include <iostream>
#include <vector>
#include <chrono>
```

```cpp
#include <iomanip>
#include <string>
using namespace std;
using namespace std::chrono;

int n, k;
vector<vector<int>> adj;
vector<int> color, primera_sol;
bool primera_guardada = false;
long long total_bt=0, total_fb=0, nodos_bt=0, nodos_fb=0;

// Escala la unidad de tiempo segun la magnitud
string formato_tiempo(double ns) {
    ostringstream oss;
    if      (ns < 1000.0)       oss << fixed << setprecision(2) << ns         << " ns";
    else if (ns < 1000000.0)    oss << fixed << setprecision(4) << ns/1000.0  << " us";
    else if (ns < 1000000000.0) oss << fixed << setprecision(4) << ns/1e6     << " ms";
    else                        oss << fixed << setprecision(4) << ns/1e9     << " s";
    return oss.str();
}

// Verifica que asignar el color c al vertice v no genere conflicto
bool es_seguro(int v, int c) {
    for (int u = 0; u < n; u++)
        if (adj[v][u] && color[u] == c) return false;
    return true;
}

// Backtracking: asigna colores vertice por vertice con poda
void bt(int v) {
    nodos_bt++;
    if (v == n) {
        total_bt++;
        if (!primera_guardada) { primera_sol=color; primera_guardada=true; }
        return;
    }
    for (int c = 1; c <= k; c++) {
        if (es_seguro(v, c)) {
            color[v] = c;
            bt(v + 1);
            color[v] = 0; // backtrack
        }
    }
}

// Verifica si una asignacion completa es una coloracion valida
```

```
bool verificar_coloracion(const vector<int>& col) {
    for (int u = 0; u < n; u++)
        for (int v = u+1; v < n; v++)
            if (adj[u][v] && col[u]==col[v]) return false;
    return true;
}

// Fuerza bruta: genera todas las k^n combinaciones y filtra las validas
void fuerza_bruta() {
    long long tc = 1;
    for (int i = 0; i < n; i++) tc *= k;
    vector<int> col(n);
    for (long long mask = 0; mask < tc; mask++) {
        nodos_fb++;
        long long tmp = mask;
        for (int v=0; v<n; v++) { col[v]=tmp%k+1; tmp/=k; }
        if (verificar_coloracion(col)) total_fb++;
    }
}

int main() {
    // Lectura del grafo
    cout << "Numero de vertices (n): "; cin >> n;
    cout << "Numero de colores  (k): "; cin >> k;
    adj.assign(n, vector<int>(n,0)); color.assign(n,0);
    cout << "Ingrese la matriz de adyacencia " << n << "x" << n << ":\n";
    for (int i=0; i<n; i++) {
        cout << "  Fila " << i << ": ";
        for (int j=0; j<n; j++) cin >> adj[i][j];
    }

    // BACKTRACKING -- medicion en nanosegundos
    auto t0 = high_resolution_clock::now();
    bt(0);
    auto t1 = high_resolution_clock::now();
    double ns_bt = duration_cast<nanoseconds>(t1-t0).count();

    cout << "\n=== BACKTRACKING ===\n";
    if (total_bt == 0)
        cout << "No existe una " << k << "-coloracion valida.\n";
    else {
        cout << "Total de coloraciones validas: " << total_bt << "\n";
        cout << "Primera solucion:\n";
        for (int v=0; v<n; v++)
            cout << "  vertice "<<v<<" -> color "<<primera_sol[v]<<"\n";
    }
    cout << "Nodos explorados: " << nodos_bt << "\n";
```

```
    cout << "Tiempo: " << formato_tiempo(ns_bt) << "\n";

    // FUERZA BRUTA -- medicion en nanosegundos
    auto t2 = high_resolution_clock::now();
    fuerza_bruta();
    auto t3 = high_resolution_clock::now();
    double ns_fb = duration_cast<nanoseconds>(t3-t2).count();

    cout << "\n=== FUERZA BRUTA ===\n";
    cout << "Combinaciones generadas: " << nodos_fb
         << " (" << k << "^" << n << ")\n";
    cout << "Total de coloraciones validas: " << total_fb << "\n";
    cout << "Tiempo: " << formato_tiempo(ns_fb) << "\n";

    // Verificacion y comparacion
    cout << "\n=== VERIFICACION CRUZADA ===\n";
    cout << (total_bt==total_fb ? "OK" : "ERROR")
         << "  Ambos: " << total_bt << " coloraciones\n";

    cout << "\n=== COMPARACION ===\n";
    double r = (1.0-(double)nodos_bt/nodos_fb)*100.0;
    cout << "Backtracking: "<<nodos_bt<<" nodos | "<<formato_tiempo
       (ns_bt)<<"\n";
    cout << "Fuerza Bruta: "<<nodos_fb<<" nodos | "<<formato_tiempo
       (ns_fb)<<"\n";
    cout << fixed << setprecision(2)
         << "Reduccion de nodos: " << r << "%\n";
    return 0;
}
```

Listing 3: main.cpp — Coloracion de Grafos

5. Ejemplos de Entrada y Salida

5.1. Ejemplo 1 — Ciclo C4 (n=4, k=3)

El ciclo C_4 es un cuadrado: 4 vertices donde cada uno esta conectado con sus dos vecinos inmediatos. Su numero cromatico es $\chi(C_4) = 2$ (grafo bipartito), por lo que con $k = 3$ existen muchas soluciones validas.

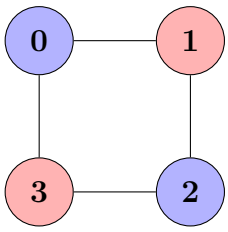


Figura 2: Primera solucion para C_4: colores alternados 1-2-1-2.

Entrada:

```
4
3
0 1 0 1
1 0 1 0
0 1 0 1
1 0 1 0
```

Salida:

```
=== BACKTRACKING ===
Total de coloraciones validas: 18
Primera solucion:
  vertice 0 -> color 1
  vertice 1 -> color 2
  vertice 2 -> color 1
  vertice 3 -> color 2
Nodos explorados: 40
Tiempo: 2.0300 us

=== FUERZA BRUTA ===
Combinaciones generadas: 81 (3^4)
Total de coloraciones validas: 18
Tiempo: 3.0610 us

=== VERIFICACION CRUZADA ===
OK  Ambos: 18 coloraciones

=== COMPARACION ===
Backtracking: 40 nodos | 2.0300 us
Fuerza Bruta: 81 nodos | 3.0610 us
Reduccion de nodos: 50.62%
```

5.2. Ejemplo 2 — Triangulo K3 (n=3, k=2) — Sin solucion

El triangulo requiere al menos 3 colores ($\chi(K_3) = 3$). Con $k = 2$ el backtracking lo descubre rapidamente (solo 5 nodos), mientras que la fuerza bruta recorre todas las $2^3 = 8$ combinaciones antes de concluir.

Entrada:

```
3
2
0 1 1
1 0 1
1 1 0
```

Salida:

```
=== BACKTRACKING ===
No existe una 2-coloracion valida.
Nodos explorados: 5
```

```
Tiempo: 513.00 ns

=== FUERZA BRUTA ===
Combinaciones generadas: 8 (2^3)
Total de coloraciones validas: 0
Tiempo: 563.00 ns
```

5.3. Ejemplo 3 — Grafo completo K7 (n=7, k=4) — Diferencia dramatica

Este caso ilustra de forma clara la ventaja del backtracking. El grafo K_7 tiene 7 vertices todos conectados entre si, por lo que $\chi(K_7) = 7$ y con $k = 4$ no existe ninguna coloracion valida. El backtracking lo determina explorando solo 65 nodos, mientras que la fuerza bruta genera las $4^7 = 16\,384$ combinaciones completas.

Entrada:

```
7
4
0 1 1 1 1 1 1
1 0 1 1 1 1 1
1 1 0 1 1 1 1
1 1 1 0 1 1 1
1 1 1 1 0 1 1
1 1 1 1 1 0 1
1 1 1 1 1 1 0
```

Salida:

```
=== BACKTRACKING ===
No existe una 4-coloracion valida.
Nodos explorados: 65
Tiempo: 5.2680 us

=== FUERZA BRUTA ===
Combinaciones generadas: 16384 (4^7)
Total de coloraciones validas: 0
Tiempo: 548.8720 us

=== COMPARACION ===
Backtracking:    65 nodos |   5.2680 us
Fuerza Bruta: 16384 nodos | 548.8720 us
Reduccion de nodos: 99.60%
```

6. Comparacion de Tiempos de Ejecucion

Se midieron tres grafos con caracteristicas bien distintas para mostrar como varia el rendimiento segun la densidad y el tamano del grafo. Los tiempos se midieron en nanosegundos y se expresan en la unidad mas legible segun la magnitud.

Cuadro 2: Comparacion de nodos explorados y tiempos de ejecucion

Grafo	**n / k**	**Nodos BT**	**Nodos FB**	**T. BT**	**T. FB**
Ciclo C_4	n=4, k=3	40	81	2.03 us	3.06 us
Triangulo K_3	n=3, k=3	16	27	1.07 us	1.21 us
Completo K_7	n=7, k=4	65	16384	5.27 us	548.87 us

Analisis de los resultados:

- En el ciclo C_4, el backtracking exploro 40 nodos frente a los 81 de la fuerza bruta, una reduccion del **50.6 %**. Los tiempos son del orden de microsegundos y bastante similares porque el grafo es pequeno; la diferencia es mas visible en el conteo de nodos.
- En el triangulo K_3 con $k = 3$, el backtracking exploro 16 nodos frente a 27, una reduccion del **40.7 %**. Aqui la poda es moderada porque hay soluciones: el backtracking encuentra rapidamente ramas validas y no necesita podar tanto.
- El caso mas revelador es K_7 con $k = 4$, donde no existe ninguna solucion valida ($\chi(K_7) = 7 > 4$). El backtracking exploro apenas **65 nodos** porque detecta el conflicto desde los primeros niveles y poda el arbol agresivamente. La fuerza bruta, en cambio, genero las $4^7 =$ **16 384** combinaciones completas sin ningun atajo, resultando en una reduccion del **99.6 %** de nodos y una diferencia de tiempo de mas de **100x** (5.27 us vs 548.87 us).

Este comportamiento es consistente con la teoria: cuanto mas denso es el grafo y mas restrictivo es el problema (pocos colores, muchas aristas), mayor es el efecto de la poda del backtracking.

7. Analisis de Complejidad

7.1. Complejidad Temporal

7.1.1. Fuerza Bruta

La fuerza bruta genera exactamente k^n combinaciones y verifica cada una recorriendo todas las aristas, que en el peor caso (grafo completo) son $n(n-1)/2 = O(n^2)$:

$$T_{\text{FB}}(n, k) = O(k^n \cdot n^2) \tag{2}$$

Este es siempre el mismo costo sin importar la estructura del grafo.

7.1.2. Backtracking

En el peor caso (grafo sin aristas: nunca hay conflictos, nunca se poda), el backtracking construye el arbol de decision completo con factor de ramificacion k y profundidad n:

$$T_{\text{BT}}^{\text{peor}}(n, k) = O(k^n) \tag{3}$$

En la practica, para grafos densos o con muchas restricciones, la poda descarta ramas enteras desde los primeros niveles. En el caso extremo de K_7 con $k = 4$, el backtracking visito solo 65 de los $4^7 = 16\,384$ nodos posibles, una reduccion del 99.6 %.

La diferencia clave entre los dos enfoques es que la fuerza bruta siempre paga el costo completo $O(k^n \cdot n^2)$, mientras que el backtracking puede ser exponencialmente mas rapido cuando la poda es efectiva.

7.2. Complejidad Espacial

Cuadro 3: Complejidad espacial por componente

Estructura	**Backtracking**	**Fuerza Bruta**
Matriz de adyacencia	$O(n^2)$	$O(n^2)$
Arreglo de colores	$O(n)$	$O(n)$
Pila de recursion	$O(n)$	$O(1)$
Total	$\mathbf{O(n^2)}$	$\mathbf{O(n^2)}$

Ambos enfoques tienen la misma complejidad espacial $O(n^2)$, dominada por la matriz de adyacencia. La pila de recursion del backtracking agrega $O(n)$ adicional, pero este termino queda subordinado a $O(n^2)$ para $n \geq 2$.

8. Respuestas a las Preguntas Guia

8.1. ¿Cuantos nodos explora el Backtracking vs la Fuerza Bruta?

En el ciclo C_4 con $k = 3$:

- **Fuerza Bruta:** $3^4 = 81$ nodos (todas las combinaciones).
- **Backtracking:** 40 nodos (50.6 % menos).

La razon es que el backtracking descarta ramas en cuanto detecta un conflicto. Si el vertice 0 tiene color 1 y el vertice 1 (adyacente) intenta tomar tambien color 1, se poda de inmediato, evitando explorar los $3^2 = 9$ sub-estados restantes de esa rama. En grafos mas densos como K_7, el efecto se amplifica hasta una reduccion del 99.6 %.

8.2. ¿Que tipo de grafo maximiza o minimiza la poda?

- **Maximiza la poda — grafo denso (K_n):** con muchas aristas casi cualquier asignacion genera conflictos desde el primer vertice, por lo que se eliminan sub-arboles muy grandes de forma temprana. En el caso de K_7 con $k = 4$, el backtracking exploro solo 65 nodos de los 16 384 posibles.
- **Minimiza la poda — grafo sin aristas:** como no existe ninguna restriccion de vecindad, `es_seguro` siempre retorna verdadero y el backtracking recorre el arbol completo igual que la fuerza bruta, sin poder eliminar ninguna rama.

8.3. ¿Que cambio hacer para saber si existe al menos una k-coloracion?

Basta con retornar en cuanto se encuentre la primera solucion, propagando un valor de exito hacia arriba:

```
bt_existe(v):
  si v == n:
    retornar verdadero        // primera solucion hallada
  para c en [1..k]:
    si es_seguro(v, c):
      color[v] = c
      si bt_existe(v + 1):
        retornar verdadero    // propagar el exito sin seguir
           buscando
      color[v] = 0
  retornar falso
```

Esto convierte el algoritmo de enumeracion en uno de decision: en el mejor caso termina mucho antes de explorar todas las ramas.

8.4. Relacion entre la k-coloracion y el Numero Cromatico

El numero cromatico $\chi(G)$ es el minimo k para el que existe una k-coloracion valida:

$$\text{El grafo } G \text{ admite } k\text{-coloracion valida} \iff \chi(G) \leq k \tag{4}$$

El algoritmo de este ejercicio permite calcular $\chi(G)$ ejecutandolo para $k = 1, 2, 3, \ldots$ e informando el primer k para el que `total_bt` > 0. Valores canonicos conocidos:

- $\chi(C_{2m}) = 2$ (ciclos pares); $\chi(C_{2m+1}) = 3$ (ciclos impares).
- $\chi(K_n) = n$ (grafo completo).
- Todo grafo planar: $\chi \leq 4$ [1].

9. Conclusiones

1. El backtracking y la fuerza bruta producen el mismo numero de coloraciones validas, confirmado por la verificacion cruzada en todos los casos probados.
2. El backtracking es significativamente mas eficiente gracias a la poda: redujo los nodos explorados un 50.6 % en C_4, un 40.7 % en K_3 y un 99.6 % en K_7. La ventaja crece con la densidad del grafo.
3. Cuando el grafo no tiene aristas el backtracking no puede podar ninguna rama y su rendimiento es identico al de la fuerza bruta.
4. La medicion de tiempos en nanosegundos con escalado automatico de unidad (ns/us/ms/s) es fundamental para comparar algoritmos sobre instancias pequenas, donde los tiempos en milisegundos simplemente muestran cero.

5. La complejidad peor caso de ambos enfoques es exponencial en n, por lo que su uso practico se limita a grafos de pocos vertices. Para instancias grandes existen heuristicas y algoritmos de aproximacion mas apropiados.
6. La relacion entre la k-coloracion y el numero cromatico $\chi(G)$ permite usar el algoritmo para encontrar la cantidad minima de colores necesaria para colorear cualquier grafo.

Referencias

[1] K. Appel, W. Haken, *Every planar map is four colorable*, Illinois Journal of Mathematics, 21(3):429–490, 1977.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[3] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012.

[4] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.